Universidade POSITIVO

Desenvolvimento de Aplicativos Móveis

Aula 06 — Utilizando Listas

ScrollView · ListView (legado) · FlatList

Prof.: MSc. Alex Junior Nunes
pfsalexnunes@gmail.com

CONCEITO

Por Que Não Basta um .map() numa View?

Até agora, para repetir elementos, usamos .map() dentro de uma View ou ScrollView — e funcionou bem para poucos itens.

Poucos itens (até ~20)

View/ScrollView + .map() funciona bem. É o que já fazíamos.

Centenas ou milhares de itens

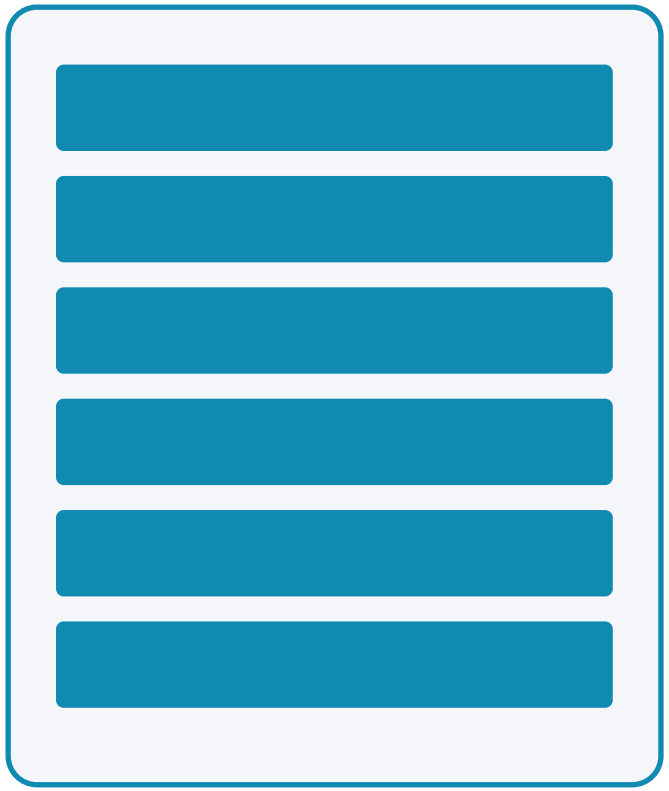
Renderizar tudo de uma vez trava o app — falta virtualização.

A solução

FlatList: desenha só o que está visível na tela, reaproveitando espaço conforme rola.

CONCEITO

ScrollView



O ScrollView desenha TODOS os itens de uma vez, estejam ou não visíveis no momento.

```
<ScrollView>
  {ITENS.map(item => (
    <Item key={item.id} {...item} />
  ))}
</ScrollView>
```

- Bom para: conteúdo curto e conhecido (tela de Sobre, formulário)
- Ruim para: listas grandes — tudo é renderizado de uma vez
- Aceita horizontal={true} para rolar na lateral

ListView — Só Contexto Histórico

O ListView foi REMOVIDO do React Native. Não existe mais nas versões atuais — não tem como rodar um exemplo dele hoje.

Antes de 2017

ListView era o único jeito de fazer listas performáticas.

2017 — FlatList chega

Mais simples de usar e mais performático — vira o padrão.

Hoje

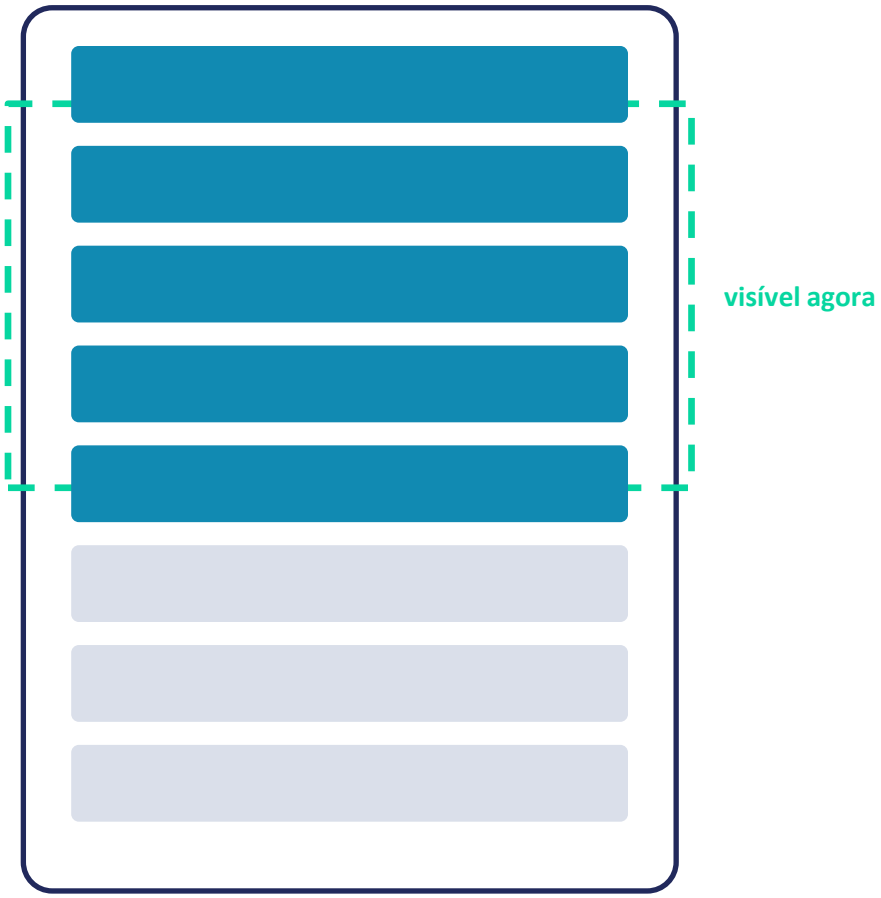
ListView não existe mais no código-fonte do React Native.

Se você ver ListView em um tutorial antigo ou pergunta de prova conceitual, saiba o que era — mas todo código novo usa FlatList.

CONCEITO

FlatList — Virtualização

O FlatList só renderiza o que está (ou está prestes a ficar) visível. O resto do array existe em memória, mas ainda não virou tela.



Itens visíveis

renderizados agora, na tela

Itens fora da tela

ainda não viraram JSX — só existem no array

É por isso que uma FlatList com 5.000 itens abre tão rápido quanto uma com 5 — o app nunca tenta desenhar os 5.000 de uma vez.

PRÁTICA

FlatList — As 3 Props Essenciais

```
<FlatList
  data={FRUTAS}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => <Text>{item.nome}</Text>}
/>
```

data	O array com os dados (aqui, sempre em memória).
renderItem	Função que recebe { item } e devolve o JSX de UMA linha.
keyExtractor	Devolve uma string única por item — obrigatório para o FlatList controlar o que já renderizou.

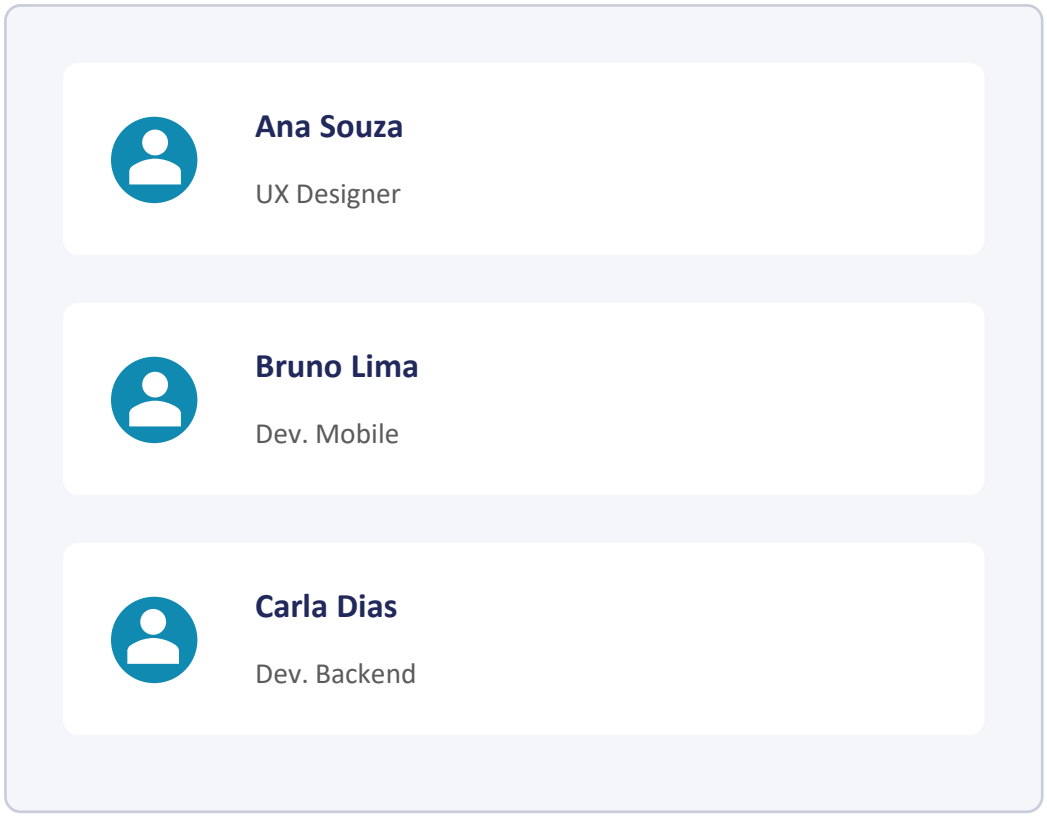
PRÁTICA

Item da Lista Como Componente

Quando a linha fica mais complexa, extraia um componente — o mesmo padrão do ItemMenu na aula de Ícones.

```
const CONTATOS = [
  { id: '1', nome: 'Ana', icone: 'user' },
  ...
];

<FlatList
  data={CONTATOS}
  keyExtractor={(i) => i.id}
  renderItem={({ item }) =>
    <ItemContato contato={item} />}
/>
```



PRÁTICA

Cabeçalho, Rodapé, Separador e Lista Vazia

`ListHeaderComponent`

um "item" fixo no topo da lista

`ListFooterComponent`

um "item" fixo no final da lista

`ItemSeparatorComponent`

desenhado ENTRE os itens (nunca antes do 1º nem depois do último)

`ListEmptyComponent`

o que mostrar quando data = []

Minhas tarefas de hoje

Estudar Flexbox

Revisar Ícones

Praticar FlatList

Fazer o desafio

4 tarefa(s) no total

PRÁTICA

FlatList Horizontal

```
<FlatList
  data={CATEGORIAS}
  horizontal  showsHorizontalScrollIndicator={false}
  renderItem={...}
/>
```

Categorias

Promoções | Novidades | Mais vendidos | Eletrônicos | ...

Só muda a prop horizontal={true} — data, renderItem e keyExtractor funcionam exatamente igual à lista vertical.

DESAFIO

Desafio da Aula: Lista de Produtos

Em duplas, monte uma tela de produtos com FlatList, usando o array PRODUTOS já pronto (sem useState):

- [ ] Cabeçalho com "Produtos" + quantidade de itens
- [ ] Cada item: ícone, nome e preço formatado (R$ 29,90)
- [ ] Um separador entre os itens
- [ ] keyExtractor usando o id do produto

`Arquivo: 05-desafio-lista-produtos.js · Gabarito: 05-desafio-lista-produtos-solucao.js`



O Que Levamos de Hoje

- ScrollView renderiza tudo de uma vez — ok para pouco conteúdo, ruim para listas grandes
- ListView foi removido do React Native — hoje é só contexto histórico
- FlatList é virtualizado: só desenha o que está (ou vai ficar) visível
- data + renderItem + keyExtractor são as 3 props que toda FlatList usa
- ListHeaderComponent, ListFooterComponent, ItemSeparatorComponent e ListEmptyComponent cobrem os casos reais de UI

Próxima aula (07): prática — melhorando um aplicativo básico já estilizado.